050709

January 29, 2026

[1]:
```
import redback
import numpy as np
import matplotlib.pyplot as plt
from astropy.cosmology import Planck18 as cosmo
import astropy.units as uu
from redback.transient_models.supernova_models import lambda_to_nu
import redback.interaction_processes as ip
import redback.sed as sed
import redback.photosphere as photosphere

import astropy.units as uu

import extinction
from extinction import ccm89, fitzpatrick99, apply, remove
from scipy.interpolate import RegularGridInterpolator
import sncosmo

def sn1998bw_template(time, redshift, amplitude, **kwargs):
    """
    A wrapper to the SN1998bw template. Only valid between 1100-11000 Angstrom␣
 ↪and 0.01 to 90 days post explosion in rest frame

    Parameters
    ----------
    time
        time in days in observer frame (post explosion)
    redshift
        redshift
    amplitude
        amplitude scaling factor, where 1.0 is the original brightness of␣
 ↪SN1998bw; and f_lambda is scaled by this factor
    kwargs
        Additional keyword arguments required by redback.
    frequency
        Required if output_format is 'flux_density'. frequency to calculate -␣
 ↪Must be same length as time array or a single number).
    bands
```

```
        Required if output_format is 'magnitude' or 'flux'.
    output_format
        'flux_density', 'magnitude', 'spectra', 'flux', 'sncosmo_source'
    cosmology
        Cosmology to use for luminosity distance calculation. Defaults to␣
↪Planck18. Must be a astropy.cosmology object.
    Returns
    -------
        set by output format - 'flux_density', 'magnitude', 'spectra', 'flux',␣
↪'sncosmo_source'

    """
    import sncosmo
    model = sncosmo.Model(source='v19-1998bw')
    original_redshift = 0.0085
    cosmology = kwargs.get("cosmology", cosmo)
    original_dl = (43*uu.Mpc).to(uu.cm).value

    # From roughly matching to Galama+ or Clocchiatti+1998bw light curves
    original_peak_time = 15
    model.set(z=original_redshift, t0=original_peak_time)
    model.set_source_peakmag(14.25, band='bessellb', magsys='ab')
    tts = np.geomspace(0.01, 90, 200)
    lls = np.linspace(1620, 11000, 300)
    f_lambda = model.flux(tts, lls) #erg/s/cm^2/Angstrom.
    l_lambda = f_lambda * 4 * np.pi * original_dl**2  # erg/s/Angstrom

    # We consider this the rest frame spectrum of 1998bw. Now we can redshift␣
↪it and scale it.
    time_obs = tts * (1 + redshift)
    lambda_obs = lls * (1 + redshift)
    dl_new = cosmology.luminosity_distance(redshift).cgs.value
    f_lambda_obs = l_lambda / (4 * np.pi * dl_new**2)
    f_lambda_obs = amplitude * f_lambda_obs * (1 + redshift) # accounting for␣
↪bandwidth stretching
    f_lambda_obs = f_lambda_obs * uu.erg / uu.s / uu.cm ** 2 / uu.Angstrom
    if kwargs['output_format'] == 'flux_density':
        frequency = kwargs['frequency']
        # work in obs frame
        ff_array = lambda_to_nu(lambda_obs)

        # Convert flux density to mJy
        fmjy = f_lambda_obs.to(uu.mJy, equivalencies=uu.
↪spectral_density(wav=lambda_obs * uu.Angstrom)).value
        # Create interpolator on obs frame grid
        flux_interpolator = RegularGridInterpolator(
            (time_obs, ff_array),
```

```
            fmjy,
            bounds_error=False,
            fill_value=0.0)

        # Prepare points for interpolation
        if isinstance(frequency, (int, float)):
            frequency = np.ones_like(time) * frequency

        # Create points for evaluation
        points = np.column_stack((time, frequency))

        # Return interpolated flux density with (1+z) correction for observer␣
 ↪frame
        return flux_interpolator(points)
    elif kwargs['output_format'] == 'spectra':
        return namedtuple('output', ['time', 'lambdas',␣
 ↪'spectra'])(time=time_obs,
                                                                          ␣
 ↪lambdas=lambda_obs,
                                                                          ␣
 ↪spectra=f_lambda_obs)
    else:
        return sed.get_correct_output_format_from_spectra(time=time,␣
 ↪time_eval=time_obs,
                                                          spectra=f_lambda_obs,␣
 ↪lambda_array=lambda_obs,
                                                          **kwargs)


# After pasting the function definition into your cell:
from redback.model_library import all_models_dict

all_models_dict['sn1998bw_template'] = sn1998bw_template

func1 = all_models_dict['sn1998bw_template']
```

```
No module named 'lalsimulation'
lalsimulation is not installed. Some EOS based models will not work. Please use
bilby eos or pass your own EOS generation class to the model
14:58 bilby INFO    : Running bilby version: 2.3.0
14:58 redback INFO    : Running redback version: 1.12.1
```

[3]:
```
#050709  *   0.161    HST/ACS                    9.800 F814W 25.8  0.1 1.0 4.
 ↪86e+40 8

time = np.array((9.800)) #days post explosion -- too early of a time ?
red = 0.161
```

```
AB_mag = 25.8 #F814W band filter -- in range ? should be !

target_frequencies = 3.73200e+14 #f814w band Hz
wavelength = np.array([8039.03000])
target_milky_way_ebv = 0.2

a_v = target_milky_way_ebv * 3.1

#WITH GAVINS IMPROVED FUNCTION:

all_models_dict['sn1998bw_template'] = sn1998bw_template

func1 = all_models_dict['sn1998bw_template']

sn_1998bw1= func1(time=time,
                  redshift=red, #redshift of actual event
                  model_kwargs=None,
                  amplitude = 1,
                  output_format='flux_density', # Returns mJy
                  frequency=target_frequencies,
                  mw_extinction=True,
                  # Add MW dust for YOUR target
                  ebv_mw=target_milky_way_ebv, # Specific to your GRB location
                  model_kwarg_names=[],
                  use_set_peak_magnitude = True,
                  peak_abs_mag = -18.6, #-18.88 B band ?
                  peak_time = 16 #shifts pak time to where it actually is
                  )

#calculate dereddened flux density of GRB evnet:
def calc_flux_density_from_ABmag(AB_mag):
    """
    Calculate flux density from AB magnitude assuming monochromatic AB filter

    :param magnitudes: AB magnitude values
    :return: flux density
    """
    return (AB_mag * uu.ABmag).to(uu.mJy)

flux_density_event = calc_flux_density_from_ABmag(AB_mag)
print(f"This is the flux density of  GRB event without extinction correction:␣
 ↪{flux_density_event:.5f}")

#de-redden flux density, account for extinction if necessary
dereddened_flux_event = extinction.remove(fitzpatrick99(wavelength, a_v, 3.
 ↪1),flux_density_event)
```

```
print(f"This is the extinction corrected GRB event flux density found using␣
 ↪fitzpatrick99: {dereddened_flux_event[0]:.5f}")

#calculate and print final ratio
flux_ratio = dereddened_flux_event / sn_1998bw1
print("The final f_98 ratio =", flux_ratio.to_value())
```

```
This is the flux density of  GRB event without extinction correction: 0.00017
mJy
This is the extinction corrected GRB event flux density found using
fitzpatrick99: 0.00024 mJy
The final f_98 ratio = [0.01497338]
```